Estandarización de la ingeniería de software, gestión del desarrollo y transición progresiva hacia entornos CI/CD: propuesta metodológica para la mejora continua institucional

Diego Alejandro Alvarado Castaño

Servicio Nacional de Aprendizaje (SENA)

Programa de Formación en Análisis y Desarrollo de Software

Artículo de revisión sistematizada y propuesta técnico-arquitectónica

14 de noviembre de 2026

Resumen

El presente artículo de investigación se desarrolla en articulación con la institución con el propósito de fortalecer la mejora continua y consolidar buenas implementaciones en sus procesos de desarrollo y despliegue de software. A partir de una revisión sistematizada de la literatura y estándares internacionales (ISO/IEC/IEEE 29148:2018 e ISO/IEC 25010:2023), se propone un modelo integral para estandarizar la gestión de requisitos, el registro de decisiones arquitectónicas (ADR), el control de versiones, la integración y la entrega continua (CI/CD). La propuesta plantea una transición gradual en cuatro etapas que prioriza la gobernanza con Git, la contenedorización mediante Docker, la revisión colaborativa de código, el fortalecimiento progresivo del equipo técnico y el diagnóstico previo de la infraestructura existente (un servidor físico y uno en la nube) antes de justificar técnica y económicamente nuevas inversiones en hardware, servicios de alojamiento (hosting) o licencias. Al tratarse de una propuesta metodológica y arquitectónica pendiente de implementación operativa, se definen indicadores DORA y métricas de calidad para su futura evaluación empírica.

Palabras clave: ingeniería de software, mejora continua, estandarización de procesos, arquitectura de software, registros de decisiones arquitectónicas, Docker, CI/CD, gestión progresiva de infraestructura.

Abstract

This research article is developed in collaboration with the institution to foster continuous improvement and consolidate sound software development and deployment practices. Based on a systematized literature review and international standards (ISO/IEC/IEEE 29148:2018 and ISO/IEC 25010:2023), a comprehensive model is proposed to standardize requirements management, architectural decision records (ADRs), version control, and continuous integration

and continuous delivery (CI/CD) workflows. The proposal outlines a four-stage gradual transition prioritizing Git governance, Docker containerization, collaborative peer code review, progressive expansion of the technical team, and a prior assessment of existing infrastructure resources (one physical server and one cloud server) before technically and economically justifying new investments in hardware, hosting services, or software licensing. Because this work constitutes a methodological and architectural proposal pending operational deployment, DORA metrics and quality indicators are established for future empirical evaluation.

Keywords: software engineering, continuous improvement, process standardization, software architecture, architectural decision records, Docker, CI/CD, progressive infrastructure management.

1. Introducción

1.1 Contexto y antecedentes

La evolución de los sistemas de información institucionales requiere articular actividades de ingeniería de requisitos, diseño arquitectónico, construcción, verificación y despliegue bajo criterios auditables y reproducibles. La ingeniería de requisitos permite transformar las necesidades operativas en especificaciones claras y verificables que guían el ciclo de vida del producto (Pacheco et al., 2018; Montgomery et al., 2022; International Organization for Standardization [ISO] et al., 2018). De manera complementaria, las decisiones sobre selección tecnológica y estructura del sistema deben responder a atributos de calidad definidos, documentando el razonamiento técnico mediante Registros de Decisiones Arquitectónicas (ADR) para preservar el conocimiento organizacional a largo plazo (Kruchten, 2004; Kopp et al., 2018; Nogueira et al., 2026).

Asimismo, la incorporación de prácticas DevOps y flujos de integración y entrega continua (CI/CD) permite automatizar la compilación, las pruebas y la liberación de nuevas versiones, fortaleciendo la estabilidad operacional (Shahin et al., 2017; Soares et al., 2022; Amaro et al., 2025). La literatura técnica señala que la transición de aplicaciones entre estaciones de desarrollo y servidores de producción requiere controles adecuados de aislamiento y configuración para evitar variaciones de comportamiento entre entornos (Nazário et al., 2025).

1.2 Planteamiento y delimitación del problema

En entornos institucionales donde el desarrollo interno de software se ha estructurado históricamente alrededor de la programación en estaciones locales y la publicación directa de funcionalidades, surgen oportunidades relevantes de mejora en materia de estandarización, trazabilidad y aseguramiento de la calidad. En el escenario analizado, la institución dispone

actualmente de un servidor físico y un servidor en la nube; sin embargo, sus funciones vigentes, capacidad disponible y posibles restricciones técnicas o administrativas aún deben ser inventariadas y evaluadas. En consecuencia, resulta prudente diagnosticar primero el estado real de dichos activos antes de asumir su disponibilidad o proyectar la adquisición de nueva infraestructura.

Con el propósito de fortalecer los procedimientos de desarrollo, integración, validación y puesta en producción, se identifican seis aspectos técnicos y operativos cuya situación actual debe verificarse mediante un diagnóstico interno:

1. **Concentración del conocimiento técnico en un solo especialista:** esquema habitual en equipos reducidos donde el dominio de la arquitectura, el código fuente y los procedimientos de publicación recae en un único ingeniero, representando una oportunidad para fomentar el respaldo colaborativo y la transferencia de conocimiento.
2. **Oportunidad de incorporar revisión cruzada del código:** ausencia de instancias formales de inspección técnica entre pares antes de integrar cambios, cuya adopción permitiría detectar de forma temprana oportunidades de optimización, mantenibilidad y seguridad.
3. **Fortalecimiento de la trazabilidad de los cambios:** necesidad de vincular sistemáticamente cada modificación en el código fuente con un requisito aprobado, una solicitud funcional o una decisión arquitectónica documentada (Mucha et al., 2024).
4. **Estandarización de mecanismos para recuperar versiones anteriores:** conveniencia de contar con artefactos versionados e inmutables que permitan ejecutar reversiones (rollbacks) rápidas y seguras ante cualquier eventualidad posterior a un despliegue.

5. **Incorporación de compuertas de validación previas a producción:** necesidad de interponer entornos de verificación y pruebas automatizadas entre la etapa de programación local y la puesta en operación.

6. **Delimitación formal entre desarrollo, pruebas y producción:** establecimiento de fronteras lógicas o físicas diferenciadas entre entornos para prevenir inconsistencias de librerías, variables o configuraciones (Nazário et al., 2025).

Delimitación del problema: Los aspectos técnicos descritos constituyen líneas de evaluación fundamentadas en la literatura de ingeniería de software para orientar el diagnóstico institucional. No representan hallazgos de una auditoría ejecutada, sino los ejes de referencia sobre los cuales se estructura la propuesta de mejora continua.

1.3 Justificación y aporte

La presente investigación se justifica por la relevancia de ofrecer a la institución una hoja de ruta técnica que permita evolucionar hacia prácticas estandarizadas de ingeniería de software de manera gradual, ordenada y sin interrumpir la continuidad operativa. El aporte principal del estudio consiste en integrar normas internacionales y evidencia empírica sobre ADR y CI/CD en un modelo de adopción progresiva. Este enfoque prioriza inicialmente el ordenamiento metodológico y el aprovechamiento responsable de la infraestructura existente, permitiendo calcular posteriormente —con base en métricas reales de operación y requerimientos del producto final— una estimación económica sustentada sobre necesidades de hardware, servicios de alojamiento (hosting) o licenciamiento.

1.4 Propósito y objetivos

Propósito: Proponer un modelo metodológico y arquitectónico de implementación gradual que apoye a la institución en su proceso de mejora continua, transitando desde un

esquema centrado en desarrollo local hacia un flujo estandarizado, colaborativo, trazable y automatizado mediante prácticas DevOps y CI/CD.

Objetivo general: Analizar y proponer buenas prácticas de ingeniería de software orientadas a estandarizar la gestión de requisitos, las decisiones arquitectónicas, el desarrollo colaborativo, la integración y el despliegue mediante pipelines CI/CD con ambientes segregados bajo un esquema de evolución progresiva.

Objetivos específicos:

1. Contrastar los aspectos operativos del desarrollo local y despliegue directo frente a soluciones estandarizadas respaldadas por normas internacionales y evidencia empírica.
2. Diseñar un esquema de gobernanza del desarrollo basado en Git, estrategia de ramas, revisión cruzada de código, contenedorización con Docker y fortalecimiento gradual del equipo técnico.
3. Definir un protocolo de diagnóstico de la infraestructura existente (servidor físico y servidor en la nube) y una arquitectura de separación controlada entre desarrollo, pruebas y producción.
4. Formular una hoja de ruta por etapas que incorpore flujos CI/CD, controles básicos de seguridad, criterios económicos para estimar hardware, hosting o licencias, e indicadores para una futura evaluación empírica.

1.5 Pregunta de investigación

¿Cómo puede fortalecerse la mejora continua en los procesos institucionales de ingeniería de software mediante la estandarización progresiva de requisitos, decisiones arquitectónicas, control de versiones, contenedorización y pipelines CI/CD con ambientes segregados sin requerir inversiones prematuras de infraestructura?

2. Marco teórico y revisión de la literatura

2.1 Ingeniería de requisitos, trazabilidad y calidad del software

En la ingeniería de software moderna, los requisitos constituyen la base técnica que define las funcionalidades del sistema (requisitos funcionales) y los atributos de calidad o restricciones operativas bajo las cuales debe operar (requisitos no funcionales). La norma ISO/IEC/IEEE 29148:2018 proporciona el marco de referencia internacional para los procesos de ingeniería de requisitos, estableciendo que cada especificación debe ser clara, completa, consistente y verificable (ISO et al., 2018). Asimismo, la revisión sistemática de Dongmo (2024) destaca que los requisitos no funcionales —tales como seguridad, rendimiento, disponibilidad y mantenibilidad— deben analizarse de manera continua a lo largo de todo el ciclo de vida del desarrollo.

Para asegurar que las modificaciones realizadas en el código respondan efectivamente a necesidades institucionales aprobadas, resulta indispensable establecer mecanismos de trazabilidad bidireccional desde las fuentes de origen hasta los casos de prueba (Mucha et al., 2024; Montgomery et al., 2022). De forma complementaria, la norma ISO/IEC 25010:2023 define el modelo de calidad del producto software estructurado en nueve características fundamentales —adecuación funcional, eficiencia de desempeño, compatibilidad, capacidad de interacción, fiabilidad, seguridad, mantenibilidad, flexibilidad y seguridad operacional—, ofreciendo criterios objetivos para orientar la mejora continua del producto y de su proceso de construcción (ISO & International Electrotechnical Commission [IEC], 2023).

2.2 Arquitectura de software y Registros de Decisiones Arquitectónicas (ADR)

La arquitectura de software comprende la organización fundamental de un sistema, sus componentes, relaciones, principios de diseño y evolución, tal como lo sistematiza la norma

ISO/IEC/IEEE 42010:2022 (ISO et al., 2022). Un desafío habitual en los equipos de ingeniería es la pérdida de contexto histórico sobre las elecciones técnicas: cuando se seleccionan herramientas, bases de datos o patrones estructurales sin registrar los criterios evaluados, resulta complejo mantener o escalar el sistema en el futuro (Kruchten, 2004; Falessi et al., 2011; Demir et al., 2024).

Para fortalecer la gestión del conocimiento técnico, la literatura recomienda el uso de Registros de Decisiones Arquitectónicas (ADR) almacenados junto al código fuente en formato de texto plano o Markdown (Nygard, 2011; Kopp et al., 2018). La investigación empírica de Nogueira et al. (2026) comparó distintas plantillas de ADR respecto a su comprensión, usabilidad y facilidad de adopción, concluyendo que la selección debe adaptarse al contexto del proyecto: la plantilla concisa de Nygard (2011) favorece la agilidad y la baja carga documental en ciclos iterativos, mientras que el formato Markdown Architectural Decision Records (MADR) propuesto por Kopp et al. (2018) aporta mayor detalle estructural para documentar alternativas evaluadas, ventajas y compromisos en decisiones arquitectónicas críticas (Zimmermann et al., 2015).

2.3 Gobernanza del código, contenedorización y prácticas DevOps / CI/CD

La integración continua (CI) consiste en integrar periódicamente los avances de desarrollo en un repositorio centralizado, ejecutando compilaciones y pruebas automatizadas para validar cada cambio de forma temprana; por su parte, la entrega continua (Continuous Delivery) garantiza que el software se mantenga en condiciones óptimas para su liberación mediante aprobación controlada (Shahin et al., 2017; Soares et al., 2022). Las revisiones sistemáticas sobre adopción de DevOps señalan que la automatización resulta más efectiva

cuando se implementa de manera gradual sobre flujos de trabajo previamente estandarizados (Laukkanen et al., 2017; Amaro et al., 2025).

Un aspecto técnico relevante documentado por Nazário et al. (2025) es la gestión de diferencias de configuración entre los entornos de desarrollo y producción (configuration drift), factor que puede provocar variaciones de ejecución al publicar una aplicación. Para asegurar la consistencia entre entornos, la literatura recomienda estandarizar el empaquetado mediante contenedores (Docker), gestionar las configuraciones de manera reproducible (Kumara et al., 2021) y aplicar controles de seguridad en el ciclo de desarrollo, conforme al marco SSDF del NIST (Souppaya et al., 2022) y el estándar IEEE 2675-2021 para construcción, empaquetado y despliegue confiable en DevOps (Institute of Electrical and Electronics Engineers [IEEE], 2021).

3. Método

3.1 Enfoque y diseño

El presente estudio adopta un enfoque cualitativo de carácter descriptivo y propositivo, estructurado mediante una revisión sistematizada de la literatura acompañada de una propuesta técnico-arquitectónica contextualizada. Este diseño resulta adecuado para fundamentar procesos institucionales de mejora continua antes de intervenir sistemas en operación, permitiendo contrastar prácticas técnicas con estándares internacionales y evidencia empírica consolidada.

3.2 Fuentes, estándares y criterios de selección

Se seleccionaron fuentes primarias bajo criterios de rigor académico, vigencia normativa y aplicabilidad directa a la ingeniería de software: (a) estándares internacionales vigentes en ingeniería de requisitos, calidad del producto, descripción arquitectónica y DevOps (ISO/IEC/IEEE 29148:2018, ISO/IEC 25010:2023, ISO/IEC/IEEE 42010:2022, IEEE 2675-2021 y NIST SP 800-218); (b) revisiones sistemáticas de literatura sobre elicitación y

trazabilidad de requisitos, integración continua, entrega continua e infraestructura como código; y (c) estudios empíricos recientes sobre adopción de plantillas ADR y estrategias de consistencia entre entornos de desarrollo y producción.

3.3 Alcance metodológico y delimitación empírica

En coherencia con el rigor científico, se indica expresamente que la propuesta descrita no ha sido implementada ni auditada operativamente en la institución tomada como contexto de estudio. Por tanto, no se reportan resultados experimentales ni mediciones operativas comprobadas. Los beneficios expuestos corresponden a mejoras proyectadas con respaldo en la literatura académica, cuya verificación efectiva requerirá ejecutar un diagnóstico inicial de infraestructura y una fase piloto controlada.

4. Resultados

4.1 Matriz comparativa de oportunidades de mejora y soluciones estandarizadas

Como primer resultado del análisis, se sistematizó la relación entre los aspectos operativos identificados en el flujo actual de desarrollo local y las prácticas estandarizadas derivadas de la revisión de literatura. La Tabla 1 presenta cada dimensión técnica por diagnosticar, su relevancia operacional y la solución propuesta para fortalecer el proceso institucional.

Tabla 1

Comparativa entre aspectos técnicos por fortalecer y soluciones estandarizadas propuestas

Dimensión	Aspecto técnico por diagnosticar	Relevancia operativa	Solución estandarizada propuesta
Control de versiones	Desarrollo concentrado en estaciones locales con oportunidad de centralizar el historial y flujo de ramas.	Permite auditar modificaciones, evitar conflictos de código y conservar trazabilidad histórica.	Repositorio Git centralizado con ramas main, develop y feature/* y protección de ramas principales.

Dimensión	Aspecto técnico por diagnosticar	Relevancia operativa	Solución estandarizada propuesta
Aseguramiento de calidad	Integración directa de cambios con oportunidad de formalizar la revisión cruzada entre pares.	Facilita la detección temprana de errores lógicos, deuda técnica y mejoras de seguridad.	Pull Requests obligatorios con revisión de al menos un par y validaciones automatizadas previas.
Conocimiento y equipo	Concentración del conocimiento técnico y operativo en un único perfil especialista.	La distribución del dominio técnico fortalece la continuidad operativa y el aprendizaje colaborativo.	Conformación gradual del equipo (ingeniero actual + al menos otro desarrollador) y uso de ADRs.
Entornos de ejecución	Configuración local heterogénea y necesidad de delimitar formalmente pruebas y producción.	Previene variaciones de comportamiento entre equipos de desarrollo y servidores (configuration drift).	Contenedorización local con Docker/Docker Compose y segregación controlada de ambientes.
Despliegue y contingencia	Publicación directa con oportunidad de estandarizar artefactos versionados y protocolos de reversión.	Agiliza la recuperación del servicio y aporta previsibilidad en cada entrega.	Pipeline CI/CD con imagen Docker inmutable versionada, secretos externos y rollback documentado.
Infraestructura y costos	Necesidad de inventariar capacidad y uso actual del servidor físico y servidor en la nube.	Optimiza el aprovechamiento de activos existentes antes de comprometer nuevas inversiones.	Diagnóstico técnico previo, uso de capacidad disponible y escalamiento progresivo por etapas.

Nota. Elaboración propia fundamentada en ISO/IEC/IEEE 29148:2018, ISO/IEC 25010:2023, Nazário et al. (2025) y Nogueira et al. (2026).

4.2 Gobernanza del desarrollo: Git, Docker y conformación progresiva del equipo

El fortalecimiento del proceso de ingeniería debe iniciar por el ordenamiento metodológico del flujo de trabajo sin exigir adquisiciones inmediatas de servidores. Para ello, se establecen tres pilares operativos:

1. **Control de versiones con Git y estrategia de ramas:** Se propone consolidar un repositorio centralizado con una estrategia de ramificación clara y auditable: main como rama protegida que refleja exclusivamente el código estable en producción; develop

como rama de integración donde convergen las funcionalidades validadas; y ramas temporales feature/* para el desarrollo aislado de cada requerimiento o mejora. Toda integración hacia develop o main debe realizarse mediante Pull Requests (PR) sujetos a revisión cruzada y aprobación formal. Es importante precisar que las ramas de Git organizan y controlan las modificaciones del código fuente, pero no constituyen por sí mismas ambientes de ejecución ni garantizan aislamiento de procesos o datos en los servidores.

2. **Estandarización de entornos locales mediante Docker y Docker Compose:** Cada desarrollador ejecutará la aplicación, sus dependencias, motores y bases de datos de desarrollo dentro de contenedores Docker orquestados mediante Docker Compose. Esta práctica homogeniza las estaciones de trabajo y agiliza la incorporación de nuevos integrantes al proyecto. No obstante, debe aclararse que los contenedores locales facilitan la reproducibilidad individual, pero no reemplazan la necesidad de contar con un ambiente centralizado de integración y pruebas.

3. **Conformación progresiva del equipo y segregación de funciones:** Con el fin de distribuir el conocimiento técnico y fortalecer la capacidad de respuesta, se recomienda conformar gradualmente un equipo de desarrollo partiendo del ingeniero actual e incorporando al menos a otro desarrollador. Esta estructura habilita la revisión cruzada de código, la documentación compartida y el registro colaborativo de decisiones arquitectónicas en formato Markdown (empleando la plantilla de Nygard para decisiones ágiles cotidianas y el formato MADR para decisiones estructurales de mayor alcance). Asimismo, deben definirse responsabilidades diferenciadas para aprobación de PR, pruebas, administración de infraestructura y autorización de despliegues: en la medida de

lo posible, quien desarrolla una funcionalidad no debe ser la única persona que la aprueba y publica en producción.

4.3 Diagnóstico de infraestructura existente y separación controlada de ambientes

Dado que la institución dispone actualmente de un servidor físico y un servidor en la nube cuyas funciones y cargas vigentes aún deben identificarse, el paso previo a cualquier decisión de inversión consiste en ejecutar un inventario técnico y operativo exhaustivo. Dicho diagnóstico debe relevar: capacidad de procesamiento (CPU), memoria RAM, almacenamiento disponible, sistema operativo, conectividad de red, servicios y aplicaciones actualmente alojados, motores de bases de datos, esquemas de copias de seguridad, responsables operativos y posibles restricciones de seguridad, licenciamiento, contratos o políticas internas que regulen el uso de estos recursos.

A partir de los hallazgos del inventario, se definen dos niveles evolutivos de arquitectura:

- **Nivel 1 — Separación mínima viable aprovechando recursos existentes:** El desarrollo se ejecuta localmente en los computadores de los programadores mediante Docker. La integración y las pruebas se alojan en recursos disponibles del servidor físico o de la infraestructura en la nube existente, siempre que el diagnóstico confirme capacidad suficiente y autorización institucional. El ambiente de producción se mantiene estrictamente aislado, operando con credenciales independientes, acceso restringido, datos protegidos, respaldos verificados y procedimientos de recuperación. Esta separación puede implementarse mediante máquinas virtuales, contenedores, redes virtuales y configuraciones independientes, sin requerir la compra inmediata de dos nuevos equipos físicos.

• **Nivel 2 — Arquitectura objetivo de mediano plazo:** Una alternativa de mayor madurez operativa consiste en disponer de dos servidores o entornos de infraestructura claramente diferenciados: uno dedicado a desarrollo, integración y pruebas, y otro destinado exclusivamente a producción (implementables mediante servidores físicos, máquinas virtuales o servicios en la nube). Debe precisarse que la separación física no es obligatoria en todos los casos; su conveniencia dependerá de la criticidad de los sistemas, los requisitos de seguridad, la carga transaccional, el presupuesto y la capacidad operativa del equipo. Asimismo, disponer de dos servidores no garantiza por sí solo alta disponibilidad, la cual requiere mecanismos adicionales de redundancia, balanceo de carga, conmutación por error y continuidad operativa.

4.4 Flujo de integración y despliegue continuo (CI/CD) y controles de seguridad

Una vez habilitado el ambiente centralizado de pruebas, se propone implementar un flujo CI/CD controlado y auditable compuesto por seis etapas secuenciales: (1) creación de una rama feature/* y ejecución de pruebas unitarias locales en contenedor; (2) apertura de un Pull Request hacia develop, disparando validaciones automatizadas (compilación, análisis estático y pruebas) junto con la revisión de código por otro integrante; (3) despliegue de la versión integrada en el ambiente de pruebas para realizar verificaciones funcionales, técnicas y de aceptación; (4) construcción de una imagen Docker versionada y promoción de ese mismo artefacto inmutable hacia producción tras aprobación autorizada, inyectando de forma independiente las variables de configuración y secretos de cada entorno; (5) registro formal del despliegue y monitoreo activo de servicios; y (6) disponibilidad de un procedimiento probado de reversión (rollback) hacia la versión estable anterior en caso de incidentes.

De manera transversal, la propuesta contempla controles básicos de seguridad operativa: no almacenar contraseñas, tokens ni archivos de secretos dentro de los repositorios Git; no utilizar bases de datos productivas para pruebas ordinarias (empleando datos sintéticos o anonimizados); restringir el acceso administrativo a los servidores; proteger las ramas principales; utilizar conexiones cifradas (HTTPS/TLS); actualizar periódicamente las dependencias; y mantener respaldos externos comprobando periódicamente su restauración.

4.5 Hoja de ruta de inversión progresiva y criterios de estimación económica

Con el objetivo de evitar cambios abruptos y compromisos financieros prematuros, la adopción del modelo se organiza en cuatro etapas progresivas. Esta secuencia permite consolidar primero los procesos de ingeniería y evaluar las necesidades reales del desarrollo finalizado antes de formular un presupuesto definitivo de hardware, servicios de alojamiento o licencias, como se resume en la Tabla 2.

Tabla 2

Etapas progresivas de implementación y criterios de estimación económica

Etapa	Alcance técnico y organizacional	Implicación económica (Hardware, Hosting y Licencias)	Condición de avance
Etapa 1: Ordenamiento base	Implementación de Git, ramas (main, develop, feature/*), Pull Requests, Docker local, ADRs iniciales e incorporación de un segundo desarrollador.	Inversión nula o mínima en infraestructura: aprovechamiento de equipos actuales y herramientas de código abierto o planes base de repositorios.	Flujo de revisión cruzada operando y entornos locales reproducibles en contenedores.
Etapa 2: Diagnóstico y pruebas	Inventario del servidor físico y nube actuales; habilitación de ambiente centralizado de integración y pruebas en capacidad disponible.	Aprovechamiento de activos existentes: sin compra de servidores nuevos, sujeto a verificación de capacidad y licencias vigentes.	Ambientes de pruebas y producción aislados lógica o físicamente con respaldos comprobados.

Etapa	Alcance técnico y organizacional	Implicación económica (Hardware, Hosting y Licencias)	Condición de avance
Etapa 3: Automatización CI/CD	Configuración de pipeline CI/CD, construcción de imágenes Docker versionadas, gestión externa de secretos, monitoreo y rollback.	Costos operativos acotados: eventual suscripción a ejecutores CI/CD, registro privado de contenedores o herramientas de observabilidad.	Artefactos inmutables promovidos entre entornos con trazabilidad y reversión validada.
Etapa 4: Escalamiento y presupuesto final	Dimensionamiento definitivo de infraestructura según la demanda real de las aplicaciones desarrolladas y métricas de operación.	Estimación económica formal: cotización sustentada de servidores dedicados o instancias cloud (CPU/RAM/disco), ancho de banda y licencias comerciales (SO, motores BD, seguridad).	Evidencia técnica de saturación de capacidad, requisitos de alta disponibilidad o cumplimiento.

Nota. La estimación económica de la Etapa 4 debe formularse únicamente tras evaluar las necesidades reales del desarrollo final y los resultados del diagnóstico de infraestructura.

5. Discusión

5.1 Interpretación técnica y organizacional de la propuesta

El análisis desarrollado muestra que la consolidación de buenas prácticas de ingeniería de software comienza por la estandarización de los flujos de trabajo y la gobernanza técnica, más que por la adquisición inmediata de nuevo equipamiento. Escalar infraestructura antes de contar con control de versiones estructurado, revisión cruzada y ambientes delimitados incrementa los costos operativos sin resolver las causas de fondo. En contraste, iniciar con Git, Docker y la conformación gradual de un equipo colaborativo permite fortalecer la trazabilidad, reducir la dependencia individual y preparar el terreno técnico para una automatización sostenible.

5.2 Comparación con la literatura previa

Los lineamientos propuestos coinciden con los hallazgos de Laukkanen et al. (2017) y Amaro et al. (2025), quienes señalan que la adopción efectiva de entrega continua requiere

alinear progresivamente los procesos de ingeniería, las pruebas y la colaboración entre roles. De igual modo, la combinación de plantillas ADR ligeras (Nygard, 2011) y estructuradas (Kopp et al., 2018) responde a la evidencia empírica de Nogueira et al. (2026), equilibrando la agilidad en equipos en crecimiento con la claridad documental necesaria para decisiones críticas. Por último, la promoción de imágenes Docker versionadas con gestión externalizada de configuraciones aplica las estrategias sistematizadas por Nazário et al. (2025) para mantener la consistencia entre desarrollo y producción.

5.3 Indicadores propuestos para la evaluación futura

Dado que aún no se han realizado mediciones empíricas dentro de la institución, resulta necesario distinguir entre los beneficios proyectados y los resultados comprobados. Para evaluar objetivamente el impacto de esta propuesta una vez iniciada su implementación, se recomienda establecer una línea base y dar seguimiento periódico a los indicadores descritos en la Tabla 3, fundamentados en el marco DORA (DORA, 2024) y criterios de aseguramiento de calidad.

Tabla 3

Indicadores propuestos para evaluar el impacto futuro del modelo en la institución

Indicador	Definición operativa	Propósito de medición
Frecuencia de despliegue	Número de versiones publicadas exitosamente en producción por unidad de tiempo.	Evaluar la cadencia y fluidez del proceso de entrega tras automatizar el pipeline.
Tiempo de entrega de cambios	Tiempo transcurrido desde la aprobación de un cambio hasta su ejecución estable en producción.	Medir la eficiencia del flujo entre desarrollo, revisión, pruebas y publicación.
Porcentaje de despliegues fallidos	Proporción de publicaciones en producción que generan incidentes o requieren corrección inmediata.	Cuantificar la efectividad de las pruebas previas y el aislamiento de ambientes.
Tiempo medio de recuperación	Tiempo necesario para restaurar el servicio tras un fallo mediante rollback o remediación.	Verificar la eficacia de los artefactos versionados y planes de contingencia.

Indicador	Definición operativa	Propósito de medición
Defectos detectados post-publicación	Cantidad de errores reportados en producción frente a los interceptados en pruebas.	Evaluar la calidad de las compuertas de validación antes del despliegue.
Cobertura de revisión y pruebas	Porcentaje de cambios integrados con Pull Request aprobado, trazabilidad y pruebas ejecutadas.	Auditar el cumplimiento real de la gobernanza técnica y colaboración del equipo.

Nota. Adaptado de las métricas de desempeño de entrega de software DORA (2024) e indicadores de aseguramiento de calidad ISO/IEC 25010:2023.

5.4 Limitaciones y líneas futuras

La principal limitación del presente trabajo radica en su alcance propositivo: al no haberse ejecutado todavía el inventario técnico de los servidores actuales ni una medición previa de tiempos de entrega, no es posible cuantificar hoy el ahorro exacto o la reducción porcentual de incidentes. Como líneas futuras se propone: (a) realizar el diagnóstico técnico y normativo del servidor físico y en la nube existentes; (b) ejecutar una implementación piloto de las Etapas 1 a 3 en un proyecto prioritario; y (c) elaborar el estudio definitivo de dimensionamiento económico (hardware, hosting y licencias) sustentado en las métricas obtenidas.

6. Conclusiones

La mejora continua de los procesos institucionales de ingeniería de software no comienza necesariamente con una inversión inmediata en infraestructura, sino con la estandarización del trabajo, la trazabilidad de requisitos y decisiones arquitectónicas, la revisión cruzada de código, la contenedorización y la separación controlada entre desarrollo, pruebas y producción. Adoptar una transición gradual —iniciando por Git, Docker y el fortalecimiento progresivo del equipo técnico— permite mitigar riesgos operativos relevantes y consolidar buenas prácticas sin generar cambios bruscos en la operación diaria.

Asimismo, el diagnóstico técnico y normativo del servidor físico y del servidor en la nube existentes constituye un paso fundamental para aprovechar de manera óptima los activos actuales antes de comprometer nuevos recursos financieros. En consecuencia, cualquier adquisición posterior de hardware, servicios de alojamiento (hosting) o licencias deberá presentarse como una decisión respaldada por evidencia empírica y por las necesidades reales del producto desarrollado, ofreciendo a la dirección una propuesta realista, técnicamente sustentada y económicamente justificable.

7. Declaraciones finales

Disponibilidad de datos y materiales: Los lineamientos metodológicos, estructuras de ramificación, esquemas de pipeline CI/CD y criterios de diagnóstico descritos en este artículo pueden ser replicados a partir de las normas internacionales y fuentes académicas citadas.

Financiamiento: Investigación académica y técnico-metodológica desarrollada de manera independiente sin financiación externa.

Conflictos de interés: El autor declara no tener conflictos de interés financieros, comerciales ni institucionales relacionados con el presente manuscrito.

Contribuciones de autoría: Diego Alejandro Alvarado Castaño: conceptualización, revisión sistematizada de literatura, análisis comparativo, diseño de la propuesta arquitectónica y redacción integral del manuscrito.

Uso de herramientas de inteligencia artificial: Se emplearon herramientas de asistencia basadas en inteligencia artificial generativa exclusivamente como apoyo para la estructuración editorial, verificación de coherencia terminológica y formateo bibliográfico bajo normas APA 7.ª edición; la selección de fuentes, el criterio técnico-arquitectónico y la validación integral del contenido fueron realizados y verificados por el autor.

8. Referencias

Amaro, R., Pereira, R., & da Silva, M. M. (2025). Mapping DevOps capabilities to the software life cycle: A systematic literature review. *Information and Software Technology, 177*, 107583. https://doi.org/10.1016/j.infsof.2024.107583

Demir, M. Ö., Chouseinoglou, O., & Kolukısa Tarhan, A. (2024). Factors affecting architectural decision-making process and challenges in software projects: An industrial survey. *Journal of Software: Evolution and Process, 36*(10), e2703. https://doi.org/10.1002/smr.2703

Dongmo, C. (2024). A review of non-functional requirements analysis throughout the SDLC. *Computers, 13*(12), 308. https://doi.org/10.3390/computers13120308

DORA. (2024). *2024 Accelerate State of DevOps report*. Google Cloud. https://dora.dev/research/2024/dora-report/

Falessi, D., Cantone, G., Kazman, R., & Kruchten, P. (2011). Decision-making techniques for software architecture design: A comparative survey. *ACM Computing Surveys, 43*(4), 1–28. https://doi.org/10.1145/1978802.1978804

Institute of Electrical and Electronics Engineers. (2021). *IEEE standard for DevOps: Building reliable and secure systems including application build, package, and deployment* (IEEE Standard No. 2675-2021). https://standards.ieee.org/ieee/43/6830/

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2018). *Systems and software engineering—Life cycle processes—Requirements engineering* (ISO/IEC/IEEE Standard No. 29148:2018). https://www.iso.org/standard/72089.html

International Organization for Standardization, International Electrotechnical Commission, & Institute of Electrical and Electronics Engineers. (2022). *Software, systems and enterprise—Architecture description* (ISO/IEC/IEEE Standard No. 42010:2022). https://www.iso.org/standard/74393.html

International Organization for Standardization, & International Electrotechnical Commission. (2023). *Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—Product quality model* (ISO/IEC Standard No. 25010:2023). https://www.iso.org/standard/78176.html

Kopp, O., Armbruster, A., & Zimmermann, O. (2018). Markdown architectural decision records: Format and tool support. En *Proceedings of the 10th Central European Workshop on Services and their Composition (ZEUS 2018)* (pp. 55–62). CEUR-WS.

Kruchten, P. (2004). An ontology of architectural design decisions in software intensive systems. En *2nd Groningen Workshop on Software Variability* (pp. 54–61).

Kumara, I., Garriga, M., Romeu, A. U., Nucci, D. D., Tamburri, D. A., Heuvel, W.-J. V. D., & Palomba, F. (2021). The do's and don'ts of infrastructure code: A systematic gray literature review. *Information and Software Technology, 137*, 106593. https://doi.org/10.1016/j.infsof.2021.106593

Laukkanen, E., Itkonen, J., & Lassenius, C. (2017). Problems, causes and solutions when adopting continuous delivery—A systematic literature review. *Information and Software Technology, 82*, 55–79. https://doi.org/10.1016/j.infsof.2016.10.001

Montgomery, L., Fucci, D., Bouraffa, A., Scholz, L., & Maalej, W. (2022). Empirical research on requirements quality: A systematic mapping study. *Requirements Engineering, 27*, 183–209. https://doi.org/10.1007/s00766-021-00367-z

Mucha, J., Kaufmann, A., & Riehle, D. (2024). A systematic literature review of pre-requirements specification traceability. *Requirements Engineering, 29*, 119–141. https://doi.org/10.1007/s00766-023-00412-z

Nazário, M., Bonifácio, R., de Souza, C., Kamei, F., & Pinto, G. (2025). Strategies to mitigate configuration differences in software development: A rapid review of grey literature. *Journal of Software Engineering Research and Development, 13*(1), 13:114–13:131. https://doi.org/10.5753/jserd.2025.4378

Nogueira, F. N., Silva, N., & Conte, T. (2026). *One size fits all? An empirical comparison of ADR templates regarding comprehension, usability, and ease of adoption*. arXiv. https://arxiv.org/abs/2604.27333

Nygard, M. (2011). *Documenting architecture decisions*. Cognitect Blog. https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Pacheco, C., García, I., & Reyes, M. (2018). Requirements elicitation techniques: A systematic literature review based on the maturity of the techniques. *IET Software, 12*(4), 365–378. https://doi.org/10.1049/iet-sen.2017.0144

Shahin, M., Babar, M. A., & Zhu, L. (2017). Continuous integration, delivery and deployment: A systematic review on approaches, tools, challenges and practices. *IEEE Access, 5*, 3909–3943. https://doi.org/10.1109/ACCESS.2017.2685629

Soares, E., Sizilio, G., Santos, J., Alencar da Costa, D., & Kulesza, U. (2022). The effects of continuous integration on software development: A systematic literature review. *Empirical Software Engineering, 27*, 78. https://doi.org/10.1007/s10664-021-10114-1

Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure Software Development Framework (SSDF) version 1.1: Recommendations for mitigating the risk of software vulnerabilities*

(NIST Special Publication 800-218). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-218

Zimmermann, O., Wegmann, L., Koziolek, H., & Goldschmidt, T. (2015). Architectural decision guidance across projects—Problem space modeling, decision backlog management and cloud computing knowledge. En *2015 12th Working IEEE/IFIP Conference on Software Architecture* (pp. 85–94). IEEE. https://doi.org/10.1109/WICSA.2015.19